

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: pd.set_option('display.max_columns', 500)
pd.set_option('display.width', 1000)
```

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [ ]: # zsq,
#data_train = pd.read_csv("/content/drive/MyDrive/(mine)Deep Learning/data_train.csv")
#data_test = pd.read_csv("/content/drive/MyDrive/(mine)Deep Learning/data_val.csv")

# xyb,
data_train = pd.read_csv("drive/MyDrive/Deep Learning Materials/Group Project/data_train.csv")
data_test = pd.read_csv("drive/MyDrive/Deep Learning Materials/Group Project/data_val.csv")
# google files
# df = pd.read_csv("data.csv")
```

```
In [ ]: data_train=data_train.drop(columns=['Unnamed: 0'])
data_test=data_test.drop(columns=['Unnamed: 0'])
```

```
In [ ]: data_train.head(5)
```

Out[]:

	temperature	age	income	Bar	CoffeeHouse	CarryAway	RestaurantLessThan20	RestaurantMoreThan20
0	0.5	0.689655	0.214286	0.125	0.125	0.125	0.125	0.125
1	1.0	0.862069	0.214286	0.125	0.750	0.250	0.250	0.250
2	1.0	0.172414	1.000000	0.000	1.000	0.750	1.000	1.000
3	1.0	0.689655	1.000000	0.125	0.250	0.250	1.000	1.000
4	1.0	0.000000	0.214286	0.250	0.250	0.125	0.250	0.250

```
In [ ]: data_train.shape
```

Out[]: (8696, 62)

```
In [ ]: data_test.head()
```

Out[]:

	temperature	age	income	Bar	CoffeeHouse	CarryAway	RestaurantLessThan20	RestaurantMoreThan20
0	1.0	0.000000	1.000000	0.125	0.250	0.250	0.250	0.250
1	1.0	0.344828	0.071429	0.750	0.000	1.000	0.250	0.250
2	1.0	0.344828	0.357143	0.250	0.250	0.750	0.250	0.250
3	1.0	1.000000	0.500000	0.000	0.125	0.125	0.250	0.250
4	0.5	0.344828	0.357143	0.000	0.125	0.750	0.125	0.125

```
Out[ ]: (2175, 62)
```

[illegible]

```

xgb_best2 = GridSearchCV(xgb_best.best_estimator_,param_grid = xgb_param_grid2,cv = 3
xgb_best2.fit(feature_train,target_train)

xgb_param_grid3 = {'gamma':range(0,20,2)}
xgb_best3 = GridSearchCV(xgb_best2.best_estimator_,param_grid = xgb_param_grid3,cv =
xgb_best3.fit(feature_train,target_train)

xgb_param_grid4 = {'subsample':[0.6,0.7,0.75,0.8,0.85,0.9]}
xgb_best4 = GridSearchCV(xgb_best3.best_estimator_,param_grid = xgb_param_grid4,cv =
xgb_best4.fit(feature_train,target_train)

xgb_param_grid5 = {'learning_rate':[0.001,0.01,0.1,0.25]}
xgb_best5 = GridSearchCV(xgb_best4.best_estimator_,param_grid = xgb_param_grid5,cv =
xgb_best5.fit(feature_train,target_train)

xgb_best_end=xgb_best5

print(xgb_best_end.best_estimator_)
print(xgb_best_end.best_params_," ", "Score:",xgb_best_end.best_score_)
y_predict = xgb_best_end.predict(feature_test)      # 进行预测
y_prob = xgb_best_end.predict_proba(feature_test)   # 进行预测

return xgb_best_end

```

In []: xgb1 = my_xgb(X_train,Y_train,X_test,Y_test,seed=5240)

```

'''
# Result:
Fitting 3 folds for each of 6 candidates, totalling 18 fits
Fitting 3 folds for each of 15 candidates, totalling 45 fits
Fitting 3 folds for each of 10 candidates, totalling 30 fits
Fitting 3 folds for each of 6 candidates, totalling 18 fits
Fitting 3 folds for each of 4 candidates, totalling 12 fits
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None, feature_types=None,
               gamma=0, gpu_id=None, grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=0.25, max_bin=None,
               max_cat_threshold=None, max_cat_to_onehot=None,
               max_delta_step=None, max_depth=5, max_leaves=None,
               min_child_weight=100, missing=nan, monotone_constraints=None,
               n_estimators=200, n_jobs=None, num_parallel_tree=None,
               predictor=None, random_state=5240, ...)
{'learning_rate': 0.25}    Score: 0.790758993538133
'''

```

```

Fitting 3 folds for each of 6 candidates, totalling 18 fitsFitting 3 folds
for each of 15 candidates, totalling 45 fitsFitting 3 folds for each of 10
candidates, totalling 30 fitsFitting 3 folds for each of 6 candidates, tota
lling 18 fitsFitting 3 folds for each of 4 candidates, totalling 12 fitsXGB
Classifier(base_score=None, booster=None, callbacks=None,                cols
ample_bylevel=None, colsample_bynode=None,                colsample_bytree=No
ne, early_stopping_rounds=None,                enable_categorical=False, eval
_metric=None, feature_types=None,                gamma=0, gpu_id=None, grow_p
olicy=None, importance_type=None,                interaction_constraints=None
, learning_rate=0.25, max_bin=None,                max_cat_threshold=None, ma
x_cat_to_onehot=None,                max_delta_step=None, max_depth=5, max_le
aves=None,                min_child_weight=100, missing=nan, monotone_constra
ints=None,                n_estimators=200, n_jobs=None, num_parallel_tree=No
ne,                predictor=None, random_state=5240, ...){'learning_rate': 0
.25}    Score: 0.790758993538133

```

```
Out[ ]: "\n# Result:\nFitting 3 folds for each of 6 candidates, totalling 18 fits\n
Fitting 3 folds for each of 15 candidates, totalling 45 fits\nFitting 3 folds for each of 10 candidates, totalling 30 fits\nFitting 3 folds for each of 6 candidates, totalling 18 fits\nFitting 3 folds for each of 4 candidates, totalling 12 fits\nXGBClassifier(base_score=None, booster=None, callbacks=None,
      colsample_bylevel=None, colsample_bynode=None,
      colsample_bytree=None, early_stopping_rounds=None,
      enable_categorical=False, eval_metric=None, feature_types=None,
      gamma=0, gpu_id=None, grow_policy=None, importance_type=None,
      interaction_constraints=None, learning_rate=0.25, max_bin=None,
      max_cat_threshold=None, max_cat_to_onehot=None,
      max_delta_step=None, max_depth=5, max_leaves=None,
      min_child_weight=100, missing=nan, monotone_constraints=None,
      n_estimators=200, n_jobs=None, num_parallel_tree=None,
      predictor=None, random_state=5240, ...) \n{'learning_rate': 0.25}    Score: 0.790758993538133\n"
```

```
In [ ]: best_estimator = xgb1.best_estimator_
best_estimator
```

```
Out[ ]: XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
      colsample_bylevel=None, colsample_bynode=None,
      colsample_bytree=None, early_stopping_rounds=None,
      enable_categorical=False, eval_metric=None, feature_types=None,
      gamma=0, gpu_id=None, grow_policy=None, importance_type=None,
      interaction_constraints=None, learning_rate=0.25, max_bin=None,
      max_cat_threshold=None, max_cat_to_onehot=None,
      max_delta_step=None, max_depth=5, max_leaves=None,
      min_child_weight=100, missing=nan, monotone_constraints=None,
      n_estimators=200, n_jobs=None, num_parallel_tree=None,
      predictor=None, random_state=5240, ...)
```

```
In [ ]: # Copy the best XGBClassifier estimator in order to save time
best_xgb = xgb.XGBClassifier(gamma=0, learning_rate=0.25, max_depth=5, min_child_weight=10)
```

Importance provides a score that indicates how useful or valuable each feature was in the construction of the boosted decision trees within the model. The more a variable is used to make key decisions with decision trees, the higher its relative importance.

As such, feature importance can be used to interpret our data to understand the most important features that define our predictions. In this case, looking at the table above, the predictor variable that is associated with a higher number means that the variable has a higher importance in the XGBoost Classification Model.

```
In [ ]: def fit_best_xgb(XGB_Classifier, X_train, Y_train, X_test, Y_test):
# Fitting and Training the model for Train & Test sets

xgb_clf = XGB_Classifier

xgb_clf.fit(X_train, Y_train)

# Predicting using the model
trainPredictin_xgb_clf = xgb_clf.predict(X_train)
testPredictin_xgb_clf = xgb_clf.predict(X_test)

# Evaluation in test set
mse_model=mean_squared_error(Y_test, testPredictin_xgb_clf)
r2_model=r2_score(Y_test, testPredictin_xgb_clf)
# Evaluation in train seet
r2_model_train=r2_score(Y_train, trainPredictin_xgb_clf)
```

```

print('In train set, r2_model_train: ',round(r2_model_train,3))
print('In test set, r2_model: ',round(r2_model,3),' ,mse_model: ',round(mse_model,3))

# Weightage/Importance of each variable
ft_weights_xgb_clf = pd.DataFrame(xgb_clf.feature_importances_, columns=['weight'], i
ft_weights_xgb_clf.sort_values('weight',inplace=True, ascending=False)

return xgb_clf,ft_weights_xgb_clf

def myplot_feature_importance(ft_weights_xgb_clf):
    # Plotting feature importances without 0 value
    w=ft_weights_xgb_clf
    w=w.loc[w.weight!=0]

    plt.figure(figsize=(10,10))
    plt.barh(w.index, w.weight, align='center')
    plt.title("Feature importances in the XGBoost Classification model", fontsize=14)
    plt.xlabel("Feature importance")
    plt.margins(y=0.001)
    # plt.xlim(0.001, 0.15)
    plt.show()

def myplot_feature_importance0(ft_weights_xgb_clf):
    # Plotting feature importances without 0 value
    w=ft_weights_xgb_clf
    # w=w.loc[w.weight!=0]

    plt.figure(figsize=(10,10))
    plt.barh(w.index, w.weight, align='center')
    plt.title("Feature importances in the XGBoost Classification model", fontsize=14)
    plt.xlabel("Feature importance")
    plt.margins(y=0.001)
    # plt.xlim(0.001, 0.15)
    plt.show()

```

```

In [ ]: xgb_clf,ft_weights_xgb_clf = fit_best_xgb(best_xgb, X_train=X_train, Y_train=Y_train, X

In train set, r2_model_train:  0.117In test set, r2_model:  -0.039 ,mse_mo
del:  0.256

```

```

In [ ]: ft_weights_xgb_clf.T

```

```

Out[ ]:

```

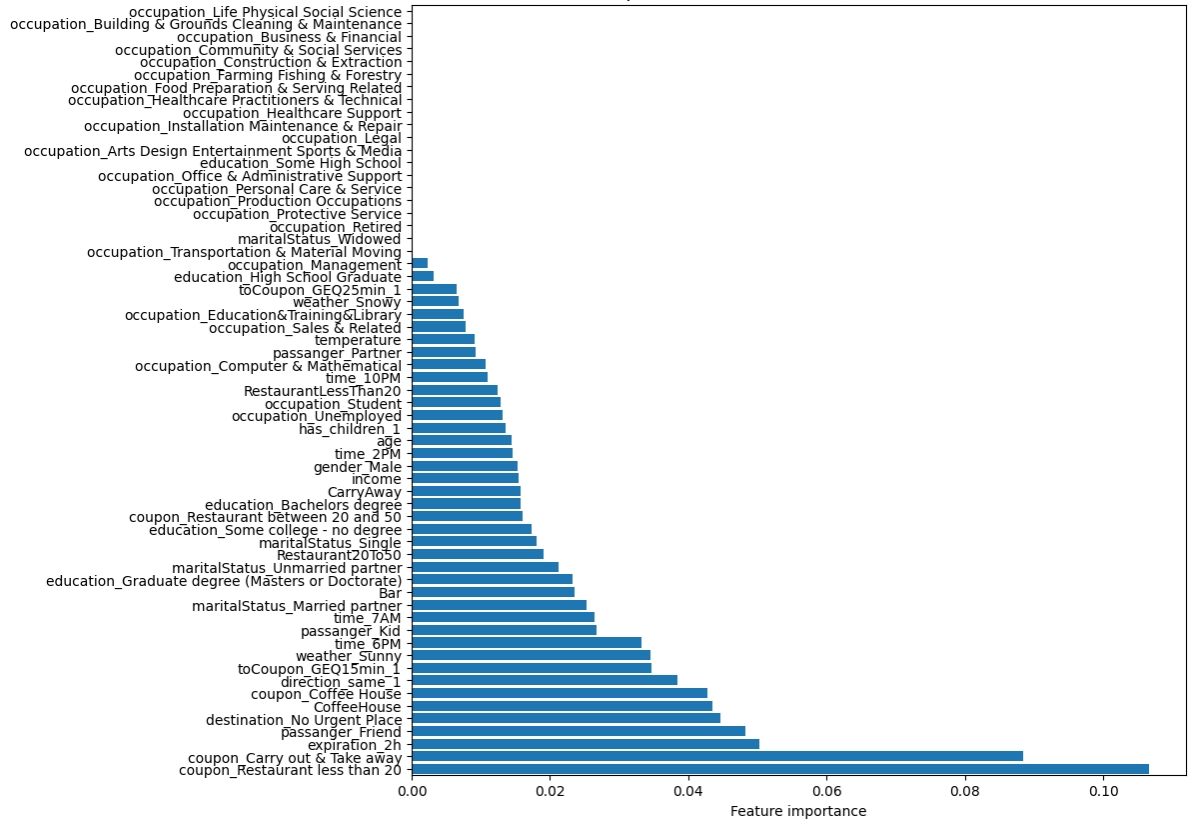
	coupon_Restaurant less than 20	coupon_Carry out & Take away	expiration_2h	passanger_Friend	destination_No Urgent Place	Coffee
weight	0.106635	0.088391	0.050301	0.048318	0.04466	0.0

```

In [ ]: myplot_feature_importance0(ft_weights_xgb_clf)

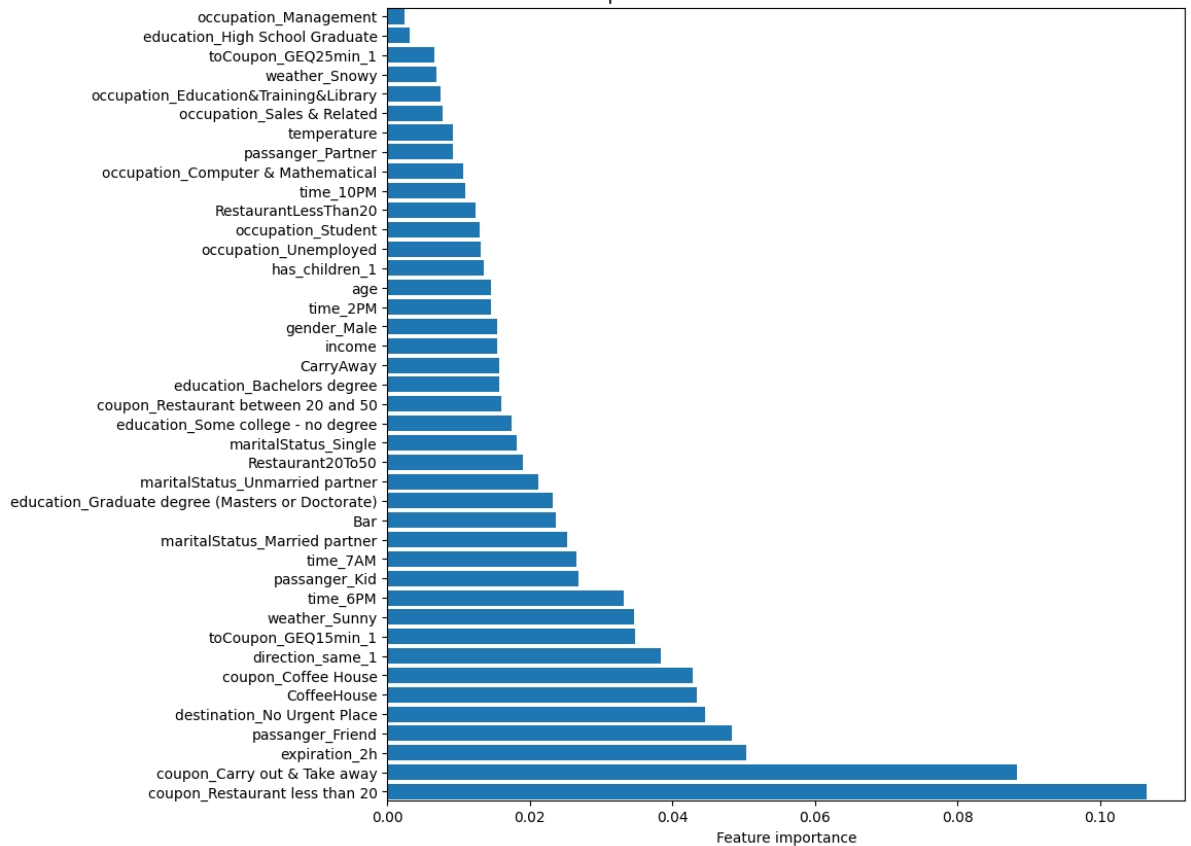
```

Feature importances in the XGBoost Classification model



```
In [ ]: myplot_feature_importance(ft_weights_xgb_clf)
```

Feature importances in the XGBoost Classification model



```
In [ ]: mask1 = (ft_weights_xgb_clf['weight']>0)
features_list = ft_weights_xgb_clf[mask1]
print('When weight>0, the number of features are ',features_list.shape[0])

mask2 = (ft_weights_xgb_clf['weight']==0)
drop_features_list = ft_weights_xgb_clf[mask2]
print('When weight==0, the number of features are ',drop_features_list.shape[0])
```

When weight>0, the number of features are 41When weight==0, the number of features are 20

```
In [ ]: features_list.T
```

```
Out[ ]:
```

	coupon_Restaurant less than 20	coupon_Carry out & Take away	expiration_2h	passanger_Friend	destination_No Urgent Place	Coffee
weight	0.106635	0.088391	0.050301	0.048318	0.04466	0.0

```
In [ ]: drop_features_list.T
```

```
Out[ ]:
```

	occupation_Transportation & Material Moving	maritalStatus_Widowed	occupation_Retired	occupation_Protecti Servi
weight	0.0	0.0	0.0	0.0

*Export to csv file

```
In [ ]: new_features_list=features_list.reset_index()  
new_features_list.rename(columns={'index': 'feature name'}, inplace=True)  
new_features_list
```

Out[]:

	feature name	weight
0	coupon_Restaurant less than 20	0.106635
1	coupon_Carry out & Take away	0.088391
2	expiration_2h	0.050301
3	passanger_Friend	0.048318
4	destination_No Urgent Place	0.044660
5	CoffeeHouse	0.043435
6	coupon_Coffee House	0.042788
7	direction_same_1	0.038425
8	toCoupon_GEQ15min_1	0.034723
9	weather_Sunny	0.034589
10	time_6PM	0.033200
11	passanger_Kid	0.026800
12	time_7AM	0.026494
13	maritalStatus_Married partner	0.025286
14	Bar	0.023614
15	education_Graduate degree (Masters or Doctorate)	0.023276
16	maritalStatus_Unmarried partner	0.021241
17	Restaurant20To50	0.019044
18	maritalStatus_Single	0.018103
19	education_Some college - no degree	0.017374
20	coupon_Restaurant between 20 and 50	0.015973
21	education_Bachelors degree	0.015763
22	CarryAway	0.015698
23	income	0.015429
24	gender_Male	0.015339
25	time_2PM	0.014566
26	age	0.014498
27	has_children_1	0.013602
28	occupation_Unemployed	0.013083
29	occupation_Student	0.012901
30	RestaurantLessThan20	0.012363
31	time_10PM	0.010923
32	occupation_Computer & Mathematical	0.010697
33	passanger_Partner	0.009232
34	temperature	0.009170
35	occupation_Sales & Related	0.007749

	feature name	weight
36	occupation_Education&Training&Library	0.007451
37	weather_Snowy	0.006838
38	toCoupon_GEQ25min_1	0.006535
39	education_High School Graduate	0.003149
40	occupation_Management	0.002343

```
In [ ]: new_features_list.to_csv('features_list_v2.csv')
```

```
In [ ]:
```